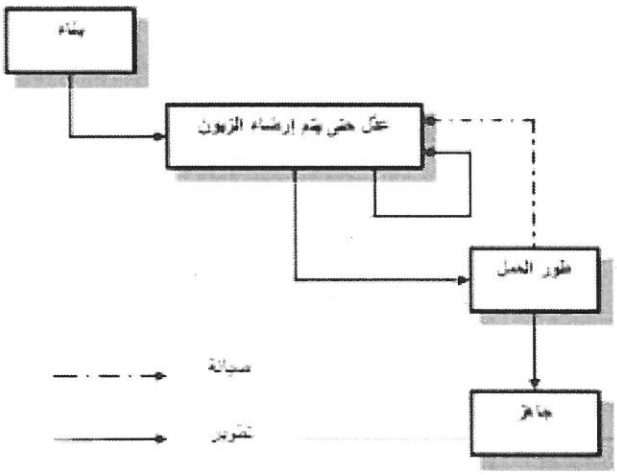
المقدمة

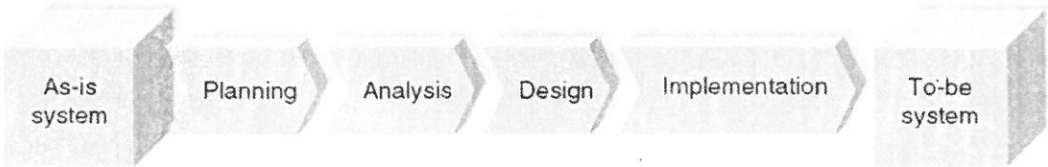
سنتناول بعض النماذج المستخدمة في اعداد المنظومات والتي ساهمت في التعامل مع الكم الهائل من المتطلبات الجديدة والمتنوعة والمعقدة والتعامل مع أزمة البرمجيات.

حيث كان في الخمسينات اعداد المنظومات يتم بطريقة بدائية تسمى **طريقة بناء-ثم-إصلاح** .أي اكتب برنامج ثم صحح الخطأ وجربه الى أن تصل الى الحل النهائي. تعمل هذه الطريقة بصورة جيدة على تمارين البرمجة ذات طول يتراوح من 100 إلى 200 سطر



ومنذ بداية السبعينات بدأ اعداد المنظومات يتم بطريقة صحيحة وهذه الطريقة تتكون من مجموعة من المراحل عرفت بإسم دورة حياة اعداد المنظومات Software Development Life Cycle **SDLC**.

في دورة حياة اعداد المنظومات يتم استخدام مجموعة نماذج مختلفة لعمليات البرمجة في هندسة البرمجيات . نموذج اعداد البرمجيات هو أسلوب لتنظيم نشاطات عمليات البرمجيات وفقا لطريقة معينة .

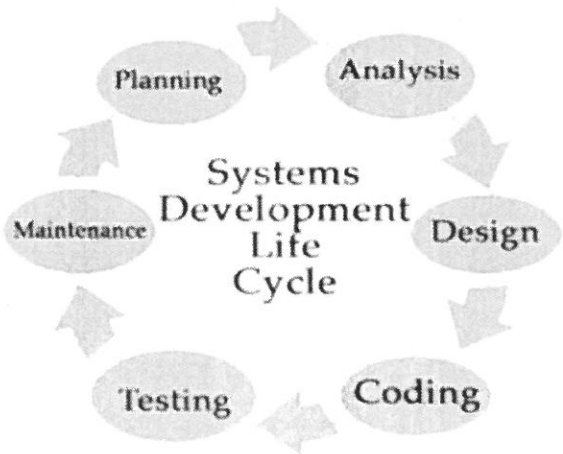


- كل نموذج هو مجموعة من االنشطة المرتبة هدفها النهائي تطوير منتج برمجي جديد أو تحسين منتج موجود.
- هذه النشاطات تهدف الي تطوير وتقييم البرمجيات .وتحتوي النشاطات على عمليات تساعد في اداء انشطة هندسة البرمجيات بطريقة منظمة.
- كل عملية من عمليات البرمجيات تتضمن العديد من الخطوات او (المهام) لكل منها مدخالتها ومخرجاتها, وتأثيرها على جودة المنتج.
- بالتالي يوجد العديد من النماذج و المنهجيات إلدارة هذه العمليات (نماذج اعداد البرمجيات)

نماذج اعداد المنظومات Software Development Models

يوجد العديد من النماذج المختلفة إلعداد المنظومات التي تم تطوريها خالل الفترة من السبعينات الي اواخر التسعينات وهي:

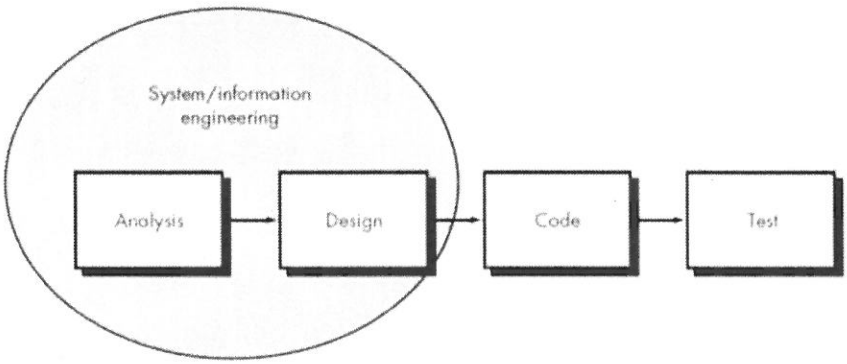
1. النموذج التدفقي أو االنحداريWaterfall Model
2. نموذج العرض التجريبيPrototype Model
3. التزايدي النموذج Incremental Model
4. النموذج اللولبي (الحلزوني) Spiral Model
5. الرشيق النموذج (Agile Model)



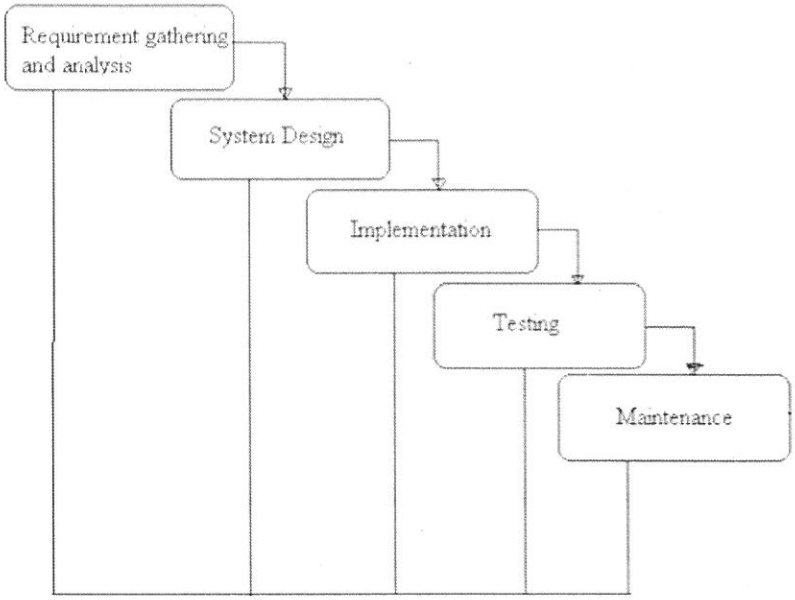
1. النموذج التدفقي Waterfall Model

يسمي النموذج الشاللي أواالنحداري أو النموذج التتابعي الخطي (Linear sequential)نظرا لتتابع المراحل أو "دورة الحياة التقليدية" هو (أول نموذج تم تتابعيا منتظما لتطوير استخدامه بداية السبعينيات) لهندسة البرمجيات يقترح منهجا البرمجيات، يبدأ عند مستوى النظام ويتقدم تباعا إلى التحليل فالتصميم فالشفرة فاالختبار فالصيانة.

يوضح دورة حياة البرمجيات بشكل تدرجي ، تبدأ من المرحلة الأولى حتي الأخيرة. كل مرحلة تبدأ بعد الإنتهاء من المرحلة السابقة.



- النموذج السابق يفترض ثبات المراحل وخاصة في مرحلة جمع المتطلبات وهي من اكثر المراحل تغيرا.
- انه لا يسمح للزبون بمشاهدة النظام إلا في المراحل المتأخرة جدا.
- ونتيجة لذلك تم تعديل في هذا النموذج واصبح **النموذج التدفقي المعدل** بحيث يمكن الرجوع للمراحل السابقة في حالة حدوث تغير في المتطلبات.



مميزات النموذج التدفقي:

1. انه بسيط وسهل الفهم.
2. يعمل بشكل جيد في المشاريع الصغيرة والتي تكون فيها المتطلبات واضحة.

عيوب النموذج التدفقي:

1. ليس نموذج جيد للمشاريع الكبيرة والمعقدة.
2. ليس جيد في المشاريع التي عادة ما تكون فيها تغير المتطلبات عالي جدا.
3. قد يسبب التكرار أو التعديلات ارتباكا (فوضى) مع تقدم فريق المشروع.

متى نستخدم النموذج الشلالي؟

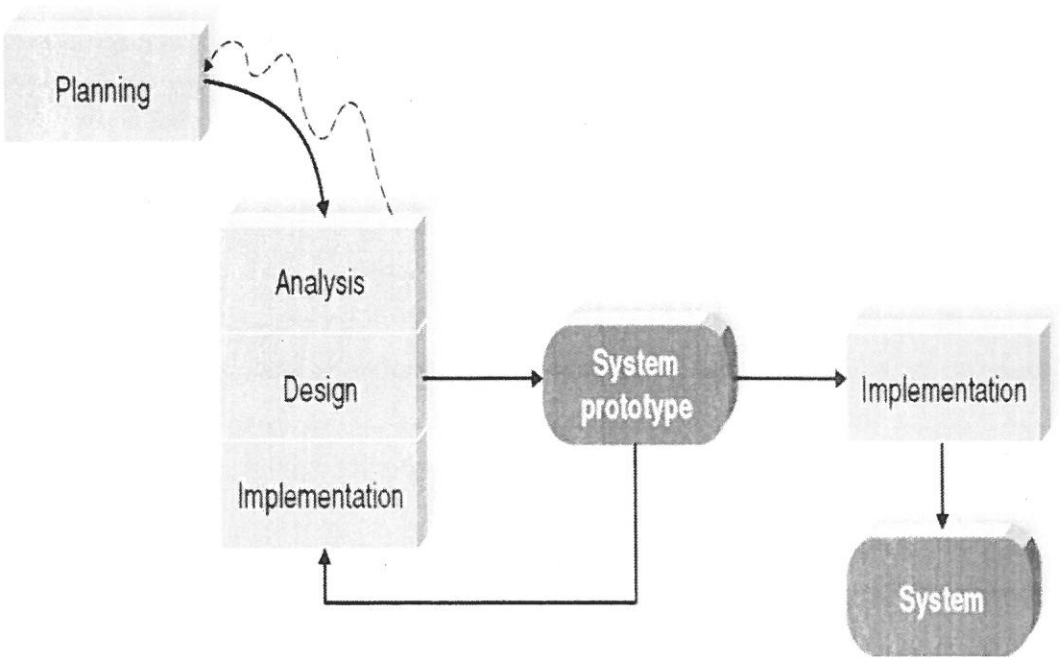
1. المتطلبات واضحة ومعرفة بشكل جيد.
2. هناك تعريف ووصف مستقر للمنتج.
3. عند بناء إصدار جديد من منتج موجود مسبقا
4. عند نقل منتج موجود إلى منصة عمل جديدة.

2. نموذج العرض التجريبي Prototype Model

جاء هذا النموذج لحل القصور في النموذج التدفقي (مشكلة المتطلبات الخاصة بالزبون الغير مدرك للمتطلبات). غالبا ما يعرف الزبون مجموعة من الأهداف العامة للبرنامج، وألا يحدد بالتفصيل كل المتطلبات.

- يقوم معد المنظومة بإعداد نموذج تجريبي (منظومة تجريبية) كعينة لعرضها على الزبون.
- تستخدم هذه العينة عادة لتوضيح لمساعدة الزبائن على فهم متطلبات النظام، وكبداية إلصدار عدة نسخ متتالية للوصول للمنظومة النهائية التي ستسلم للزبون فيما بعد.
- هذا النموذج يتم عرض وظائف محددة للمنظومة ابتداء من نسخة اولية.
- يبنى النظام كسلسلة من الإصدارات التزايدية تسلم تباعا للزبون حيث:

1. يبني المطورون **نموذج أولي (Prototype)**خالل مرحلة جمع المتطلبات.
2. يقوم المستخدمون أو الزبون بتقييم النموذج المنجز.
3. يقوم المستخدمون أو الزبون باقتراح االجراءات التصحيحية.
4. يقوم المطورون بتحسين النموذج وبناء نسخة جديدة.
5. عند بلوغ الزبون درجة مقنعة من الرضى والكفاية، يقوم المطورون بمراجعة شفرة النموذج االخير لتحسينه من خالل تطبيق المعايير اللازمة لاعتباره منتجاً نهائياً.



مميزات النموذج التجريبي:

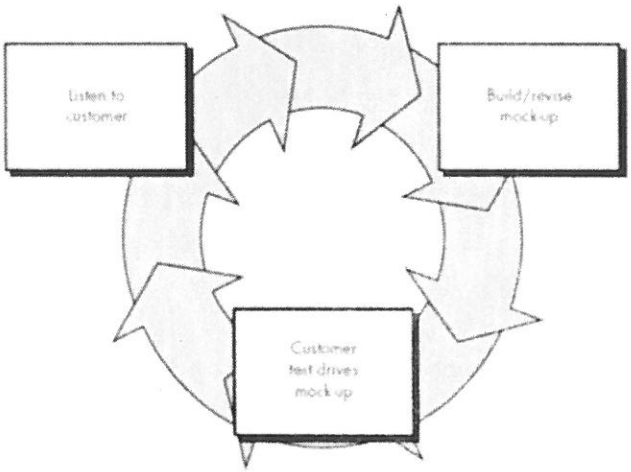
1. يقلل من مشاكل تحديد المتطلبات.
2. يوفر دعم ومشاركة اكبر من طرف الزبون في المراحل المبكرة للمشروع.
3. يستخدم عندما ال يكون هناك رؤية واضحة للمتطلبات ، بالتالي هو مناسب للبيئة التي يكثر فيها تغير المتطلبات.
4. يمكن استعماله لتدريب المستخدمين قبل التسليم

عيوب النموذج التجريبي:

1. قد يضغط الزبون باتجاه تسليم نموذج غير نهائي.
2. قد تتحول عملية تحسين الطراز إلى حلقة غير منتهية مع اكتشاف المزيد من الاحتياجات والمتطلبات(scope creep).
3. يجب أن يتمتع فريق التطوير بمهارات عالية.
4. قلة استخدام التوثيق.

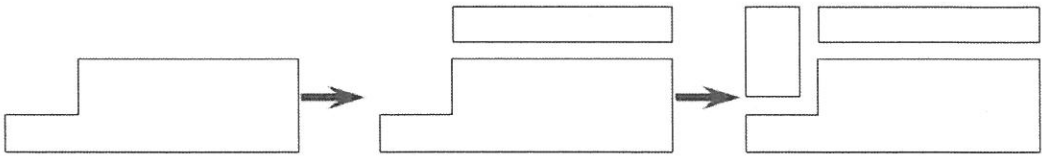
متي نستخدم العرض التجريبي؟

1. توضيح و تحديد وتنقيح المتطلبات.
2. محاولة فهم المسألة المراد حلها.
3. نسخة بدائية للتوصل الي النسخة النهائية او ما يسمي بالإصدار النهائي
4.

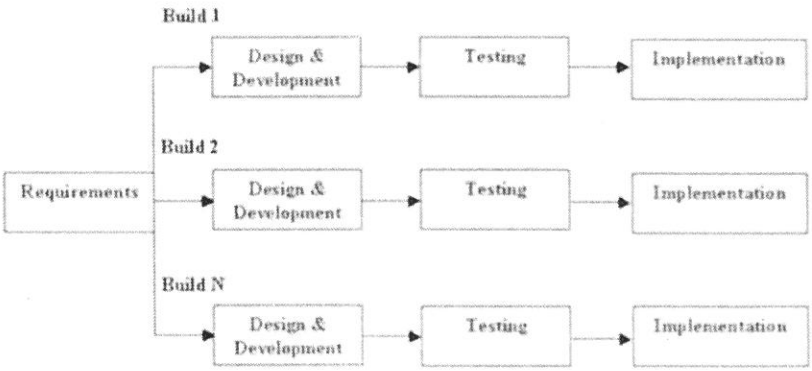


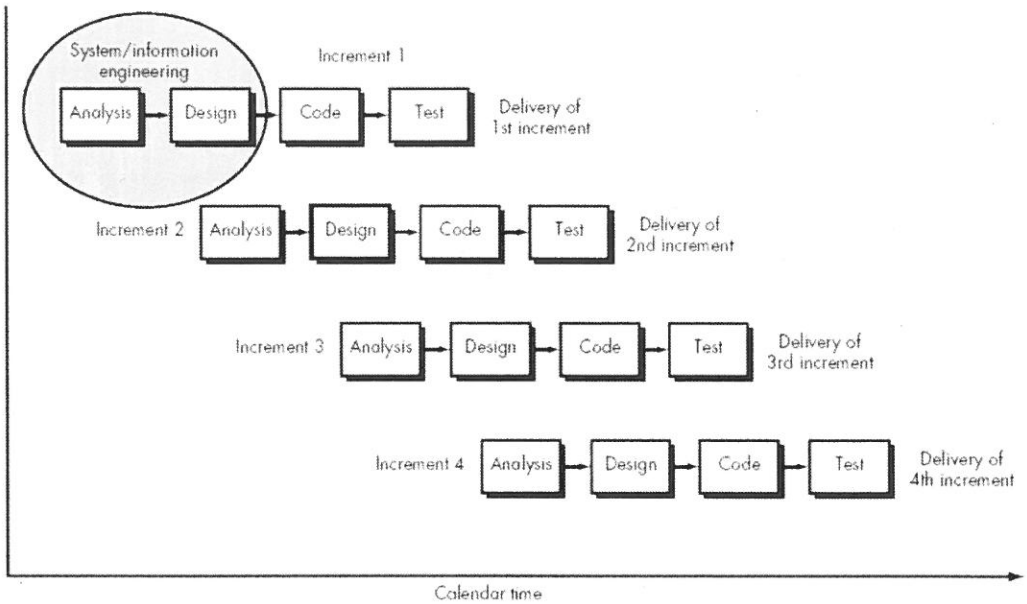
3. النموذج التزايدي Incremental Model

- الهدف من هذا النموذج هو **تقليص وقت انتظار الزبون** للحصول على المنظومة النهائية **وخاصة المنظومات الكبيرة والمعقدة** والتي تستغرق مدة طويلة لإعدادها.
- وبالتالي تسليم المنظومة علي دفعات يكون الحل. حيث يتم تقسيم المنظومة الي اجزاء أو بناء Buildيتم تسليمها للزبون تباعا في فترات زمنية متلاحقة.
- كل جزء تمر عليه جميع مراحل تطوير البرمجيات ليتم تشغيله فيما بعد في موقع الزبون.
- **النموذج التزايدي هو عبارة عن خليط من النموذج التدفقي و نموذج العرض التجريبي.**



- فمثال قد تنجز برنامج معالجة النصوص باستخدام النموذج ألتزايدي كإنشاء الملفات والتحرير في التزايد ألول، وتقدم إمكانيات أكثر تطورا (تعقيدا وتقدما) كتحرير والطباعة وادراج الجداول التزايد الثاني، وتقدم التصحيح إلمالئي والقواعد في التزايد الثالث، وتقدم إمكانات متقدمة لضبط تنسيق الصفحة(page layout) في التزايد الرابع.





خصائص النموذج التزايدي Incremental Model

1. يتكون المنتوج عادة من اجزاء أو بناء Build .
2. كل جزء(بناء) يتم تصميمه وكتابته ثم تسليمه للزبون.
3. يعطي المنتوج للزبون تدرجيا علي شكل دفعات جزء بعد جزء.

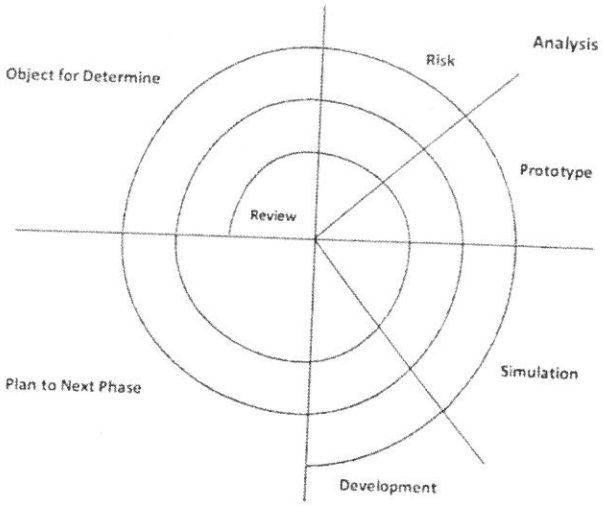
مميزات النموذج التزايدي

1. لا يحتاج الي الكثير من أعضاء فريق العمل مما يودي الي توفير التكلفة.
2. يستطيع الزبون أن يرى منظومة قابلة للعمل في وقت مبكر.
3. يمكنك تسليم االجزاء البرمجية العاجلة ، و تأجيل الباقي فيما بعد.

عيوب النموذج التزايدي

1. يحتاج الي تخطيط وتصميم جيد.
2. التكلفة أكثر من النموذج االنحداري.

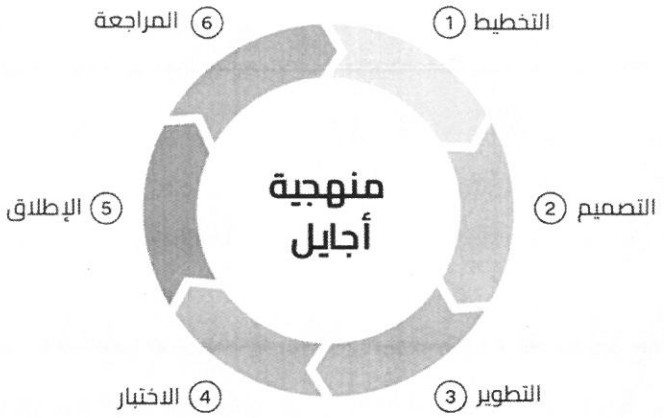
4. النموذج الحلزوني Spiral Development :



1. Object Determine: تحديد المتطلبات بعد مراجعة انظمة السوق لمعرفة ما هي الأشياء او الوظائف التي لا توجد في الانظمة الموجودة في السوق وهذه الخطوة تسميها Review.
2. Analysis:التحليل وفي هذه المرحلة نقوم بتحليل المخاطر الممكن حدوثها وذلك من خلال عمل مجموعة من الانشطة من اجل التقليل من تلك المخاطر أو الغاءها ومن المخاطر الممكن حدوثها ان يقدم رئيس الفريق استقالته او ان ينسحب احد الخبراء والمعتمد عليهم في النظام من الفريق وغيرها فتلك مخاطر لكنها ليست مخاطر على حياة الانسان وانما على عملة، ومن ثم تأتي مرحلة الـ Prototype ضمن مراحل التحليل وفي هذه الخطوة نقوم بوضع محاذاة أولية للنظام أو تصور أولي على ورق أو ان يكون ذلك التصور خوارزميات الوظائف النظام أو رسومات توضيحية.
3. Simulation: من خلال مرحلة المحاكاة نضع نموذج يعبر عن النظام او برنامج ميدني (كود برمجي) لكنه ليس نهائي للنظام.
4. Development: نبدأ هنا بتطوير النظام والمقصود كتابة الكود البرمجي.
5. Plan to Next Phase: التخطيط للمرحة القادمة أي يتم دراسة ماذا نريد أن نفعل في المرحلة التالية طالما ان النظام لم يكتمل.

وتستمر هذه المراحل بالدوران متنقلين بين انظمة النظام الفرعية حتى نصل إلى النسخة النهائية وبالتالي لن تنتقل إلى المرحلة رقم خمسة وهي التخطيط للمرحلة القادمة.

5. نموذج الرشيق (Agile Model):



تعريفه:

نموذج الأجايل هو نهج تكراري وتعاوني يهدف إلى تطوير البرمجيات بشكل مرن وسريع عبر تحسينات مستمرة بناءً على ملاحظات العميل.

مراحله (ضمن كل دورة تطوير):

1. تخطيط سريع – فهم المتطلبات الأساسية.
2. التصميم والتنفيذ – تطوير الميزات بناءً على الأولويات.
3. اختبار وتقييم – مراجعة والتحقق من جودة المنتج.
4. التفاعل مع العميل – الحصول على الملاحظات وإجراء التعديلات.
5. إعادة التكرار – تحسين المنتج بشكل مستمر.

المزايا:

1. يسمح بتسليم أجزاء وظيفية من النظام بسرعة
2. مرن في التعامل مع تغييرات المتطلبات
3. يشجع التعاون بين المطورين والعملاء

العيوب:

1. قد يكون غير مناسب للمشاريع الضخمة ذات المتطلبات الثابتة
2. يحتاج إلى فريق منظم ومتفاعل باستمرار

متى يُستخدم؟

- عندما يكون من المتوقع حدوث تغييرات كثيرة في المشروع.

- في المشاريع التي تحتاج إلى تطوير سريع وتفاعل مستمر مع العميل.

عند اختيار أفضل نهجSDLC، من المهم أن تتذكر أن أحد الحلول قد ال يناسب كل سيناريو أو عمل. قد تعمل بعض المشاريع بشكل أفضل مع نهج االنحدار ، بينما سيستفيد البعض االخر من المرونة في النماذج الرشيقة أو التكرارية.